

EduSaaS

Code Plagiarism Detection System

Updated Technical Documentation

Node.js / Express + Python / FastAPI + PostgreSQL
Supported languages: Python, JavaScript / Node.js, Java / Spring Boot

1. Purpose

The EduSaaS Code Plagiarism Detection System identifies potential similarities between student programming submissions. A submission is compared against eligible submissions for the same assignment and domain role, excluding submissions from the same user.

The current implementation is an algorithmic similarity engine, not a machine-learning classifier trained on a large plagiarism dataset. It combines Original Token Similarity, Normalized Token Similarity, and Weighted AST Similarity into a Final Similarity Score and Risk Level.

2. System Architecture

```
Student
-> Frontend
-> Node.js / Express
    |-> PostgreSQL (education schema)
    `-> Python / FastAPI
        -> Language Detection
        -> Python / JavaScript / Java analysis path
        -> Token + Normalized Token + AST similarity
        -> Plagiarism result
-> Frontend
```

3. Technology Stack

Component	Technology	Responsibility
Application API	Node.js / Express	Receives submissions, validates data, stores records, selects comparisons and calls Python.
Plagiarism Service	Python / FastAPI	Language detection, token analysis, AST analysis, scoring and risk classification.
Database	PostgreSQL	Stores submissions, checks and matches.
Node-to-Python	REST / Axios	Sends code and comparison submissions.
Python AST	Python ast	Python source parsing.
JavaScript AST	Acorn	JavaScript parsing; ES module first, script fallback.
Java AST	Tree-sitter Java	Java / Spring Boot structural parsing.
Similarity	Tokens + normalization + weighted AST	Produces similarity signals and final score.

4. Supported Domain Roles

1. AI Engineer
2. Machine Learning Engineer
3. Data Scientist
4. Data Analyst
5. Data Engineer
6. Generative AI Engineer

7. MLOps Engineer
8. Backend Developer
9. Full Stack Developer
10. Frontend Developer
11. Cloud Engineer
12. DevOps Engineer
13. Cybersecurity Analyst
14. Software Development Engineer
15. Mobile Application Developer
16. UI/UX Designer
17. Business Intelligence Developer
18. Blockchain Developer
19. IoT Engineer
20. Robotics and Computer Vision Engineer
21. Software Test Engineer

domain_role_id determines the comparison group; programming language determines the analysis path.

5. Comparison Rule

A submission is compared only when all conditions are satisfied:

- Same assignment_id
- Same domain_role_id
- Different user_id

```
SELECT submission_id, code
FROM education.final_submissions
WHERE assignment_id = $1
      AND domain_role_id = $2
      AND user_id != $3;
```

6. Database Tables

6.1 education.final_submissions

```
CREATE TABLE education.final_submissions (
  submission_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  domain_role_id UUID NOT NULL,
  assignment_id UUID NOT NULL,
  code TEXT NOT NULL,
  submitted_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_final_submission_user
    FOREIGN KEY (user_id) REFERENCES education.users(user_id),
  CONSTRAINT fk_final_submission_domain_roles
    FOREIGN KEY (domain_role_id) REFERENCES education.domain_roles(domain_role_id)
);
```

6.2 education.plagiarism_checks

```
CREATE TABLE education.plagiarism_checks (
  plagiarism_check_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  submission_id UUID NOT NULL,
  status VARCHAR(30) NOT NULL,
```

```

highest_similarity NUMERIC(5,2),
comparison_count INTEGER NOT NULL DEFAULT 0,
checked_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_plagiarism_check_submission
    FOREIGN KEY (submission_id)
    REFERENCES education.final_submissions(submission_id)
    ON DELETE CASCADE
);

```

6.3 education.plagiarism_matches

```

CREATE TABLE education.plagiarism_matches (
    match_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    plagiarism_check_id UUID NOT NULL,
    matched_submission_id UUID NOT NULL,
    original_token_similarity NUMERIC(5,2),
    normalized_token_similarity NUMERIC(5,2),
    weighted_ast_similarity NUMERIC(5,2),
    final_similarity NUMERIC(5,2),
    risk_level VARCHAR(30),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_plagiarism_match_check
        FOREIGN KEY (plagiarism_check_id)
        REFERENCES education.plagiarism_checks(plagiarism_check_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_plagiarism_match_submission
        FOREIGN KEY (matched_submission_id)
        REFERENCES education.final_submissions(submission_id)
        ON DELETE CASCADE
);

```

7. Complete Submission Workflow

22. Student submits code through the frontend.
23. Node.js validates user, domain role and assignment.
24. Node.js stores the submission in education.final_submissions.
25. Node.js selects prior submissions with the same assignment_id and domain_role_id and a different user_id.
26. Node.js calls POST /api/plagiarism/check on the Python service.
27. Python detects or receives the language.
28. Language-specific tokenization and normalization are performed.
29. Original token similarity is calculated.
30. Normalized token similarity is calculated.
31. The ORIGINAL source is parsed into the appropriate AST.
32. AST node types are extracted.
33. Language-specific AST weights are applied.
34. Weighted AST similarity is calculated.
35. Final similarity and risk are calculated.
36. Node.js stores the check and individual matches and returns the result.

7.1 Important AST Rule

Cleaned source may be used for token processing, but the original source should be supplied to AST parsers. This prevents preprocessing from destroying Python indentation/newlines or JavaScript module syntax.

8. Python Plagiarism API

Item	Value
Base URL	http://localhost:8001
Swagger	http://localhost:8001/docs
Endpoint	POST /api/plagiarism/check

8.1 Request

```
{
  "submission": {
    "submission_id": "550e8400-e29b-41d4-a716-446655440001",
    "code": "def calculate_sum(a, b):\n    return a + b"
  },
  "comparison_submissions": [
    {
      "submission_id": "550e8400-e29b-41d4-a716-446655440002",
      "code": "def add_values(x, y):\n    return x + y"
    }
  ]
}
```

8.2 Response

```
{
  "submission_id": "550e8400-e29b-41d4-a716-446655440001",
  "comparison_count": 1,
  "matches": [{
    "submission_id": "550e8400-e29b-41d4-a716-446655440002",
    "original_token_similarity": 57.89,
    "normalized_token_similarity": 100.0,
    "weighted_ast_similarity": 100.0,
    "final_similarity": 93.68,
    "risk_level": "VERY_HIGH"
  }]
}
```

9. Multi-Language Analysis Pipeline

```
Source
-> Language Detector
    -> Python          -> Python tokenizer/normalizer -> Python AST
    -> JavaScript      -> JS tokenizer/normalizer     -> Acorn AST
    -> Java            -> Java tokenizer/normalizer    -> Tree-sitter AST
-> Original Token Similarity
-> Normalized Token Similarity
-> Weighted AST Similarity
-> Final Similarity
-> Risk Level
```

10. Similarity Engine Modules

Module	Purpose
language_detector.py	Detects Python, JavaScript or Java.
token_similarity.py	Generic tokenization and original token similarity.
token_normalizer.py	JavaScript normalization.
python_token_normalizer.py	Python normalization.
java_token_normalizer.py	Java tokenization/normalization; preserves Java/Spring constructs.
normalized_token_similarity.py	60% frequency + 40% sequence similarity.
python_ast.py	Python AST parsing.

javascript_ast.py	Invokes Acorn parser.
java_ast.py	Tree-sitter Java parsing.
ast_features.py / python_ast_features.py / java_ast_features.py	AST node fingerprints.
weighted_ast.py / python_weights.py / java_ast_weights.py	Language-specific AST weights.
weighted_ast_similarity.py	Weighted structural similarity.
score_calculator.py	Final score and risk.
plagiarism_service.py	Coordinates all language-specific paths.
ast_parser/parser.js	Acorn parser; module first, script fallback.

11. Original Token Similarity

Original Token Similarity compares actual source tokens. Identifier changes can lower this score. It is a similarity measure, not a percentage of text proven to be copied.

12. Normalized Token Similarity

Normalized Token Similarity reduces the effect of identifier renaming. Language-specific normalizers convert user-defined identifiers to stable placeholders while preserving important keywords, syntax and framework constructs.

```
Student A:
model.fit(X_train, y_train)

Normalized:
VAR0.fit(VAR1, VAR2)

Student B:
classifier.fit(train_data, train_target)

Normalized:
VAR0.fit(VAR1, VAR2)
```

The current normalized-token score combines token-frequency similarity (60%) and token-sequence similarity (40%).

13. AST Similarity

AST analysis compares structural organization rather than only textual spelling.

13.1 Python

- Python ast.parse()
- Python node-type fingerprint
- Python-specific weights

13.2 JavaScript / Node.js

- Acorn
- ES module parsing first
- Script fallback
- JavaScript node-type fingerprint

- JavaScript weights

13.3 Java / Spring Boot

- Tree-sitter Java
- Java node-type fingerprint
- Java-specific weights
- Java token normalization
- Spring annotations/framework constructs preserved by the Java normalizer

14. Weighted AST Similarity

Structural nodes can carry more information than simple identifiers or constants. The weighted algorithm uses configured node weights and compares common versus maximum node counts.

```
common_weight += min(count_a, count_b) * weight
total_weight += max(count_a, count_b) * weight

score = (common_weight / total_weight) * 100
```

15. Final Similarity Score

Signal	Weight
Original Token Similarity	15%
Normalized Token Similarity	25%
Weighted AST Similarity	60%

AST is currently the dominant signal. These weights should be calibrated using labeled evaluation data before production use.

16. Risk Classification

Final Score	Risk
0–29.99	LOW
30–59.99	MEDIUM
60–79.99	HIGH
80–100	VERY_HIGH

Risk is an indicator for human review, not automatic proof of academic misconduct.

17. JavaScript Parser Update

The earlier parser used `sourceType='commonjs'`, which failed for `import/export` syntax. The updated Acorn parser attempts ES-module parsing first and falls back to script parsing.

```
const ast = acorn.parse(source, {
  ecmaVersion: "latest",
  sourceType: "module",
  locations: true,
  ranges: true,
  allowHashBang: true
});
```

If module parsing fails, the parser retries with `sourceType='script'`.

18. Java / Spring Boot Validation Tests

18.1 Highly Similar / Renamed Implementation

Metric	Observed
Original Token	59.32%
Normalized Token	100.00%
Weighted AST	100.00%
Final	93.90%
Risk	VERY_HIGH

The two implementations had substantially the same Spring Boot structure while changing identifiers and names. The detector therefore produced a very high similarity signal.

18.2 Independent Implementation

An independently structured solution used Map-based input/output and a different implementation structure.

Metric	Observed
Original Token	35.71%
Normalized Token	48.74%
Weighted AST	39.35%
Final	41.15%
Risk	MEDIUM

This is the desired comparative behavior for the current test: solving the same assignment does not automatically result in a very-high similarity when the implementation structure is substantially different.

19. Previous ML / XGBoost Tests

Test	Original	Normalized	AST	Final	Risk
Highly similar implementation	66.20%	97.72%	98.66%	93.56%	VERY_HIGH
More independent implementation	55.26%	76.70%	62.75%	65.11%	HIGH

20. First Submission Handling

If no eligible comparison exists, the submission is still stored.

```
{
  "status": "NO_COMPARISONS",
  "highest_similarity": 0,
  "comparison_count": 0,
  "matches": []
}
```

21. Error Handling

Error	Cause / Action
404 Route not found	Verify the requested Node.js route and HTTP method.
Plagiarism service unavailable	Start FastAPI and verify the configured Python

	service URL/port.
JavaScript parsing error	Check Acorn syntax support and module/script parsing.
Python parsing error	Ensure original Python source is used for AST parsing.
Java parsing error	Verify Java syntax and Tree-sitter Java setup.
Language mismatch	Use the same programming language for both submissions.

22. Evaluation and Calibration

Before production deployment, evaluate the detector with labeled examples:

- Exact copies
- Variable-renamed copies
- Reformatted copies
- Modified copies
- Partially copied implementations
- Independent implementations of the same assignment
- Unrelated implementations
- Framework-specific implementations

Recommended metrics: Precision, Recall, F1-score, False Positive Rate and False Negative Rate.

Thresholds and weights should be calibrated from the evaluation results.

23. Current Limitations

- The detector is an algorithmic similarity engine, not a trained ML/DL classifier.
- High similarity is an indicator for review, not proof of misconduct.
- Common assignment boilerplate can create legitimate structural similarity.
- Semantic equivalence can exist even when token/AST similarity is low.
- Obfuscation and generated code may require additional analysis.
- Additional languages require dedicated tokenizers and parsers.

24. Security and Privacy

- Authenticate and authorize users before accepting submissions.
- Authorize access to plagiarism results.
- Do not expose other students' source code or identity information.
- Use HTTPS in production.
- Store database credentials in environment variables/secrets.
- Restrict direct public access to the Python service.
- Avoid logging complete student source code unnecessarily.

25. Deployment Checklist

37. Create the three education plagiarism tables.
38. Configure PostgreSQL for Node.js.
39. Start the FastAPI service on the configured port.
40. Install Acorn and Node dependencies.
41. Verify Tree-sitter Java dependencies.
42. Test POST /api/plagiarism/check using Swagger/Postman.
43. Test Node.js submission flow.
44. Verify same assignment + same role + different user filtering.
45. Test first-submission behavior.
46. Test exact, renamed and independent implementations.
47. Calibrate thresholds with labeled evaluation data.

26. Conclusion

The updated EduSaaS detector supports Python, JavaScript/Node.js and Java/Spring Boot using language-specific tokenization, normalization and AST analysis. Comparison-group selection is separated from code analysis: assignment_id, domain_role_id and user_id determine which submissions are compared, while language determines how code is analyzed.

Current validation demonstrates useful separation between a renamed/highly similar Java implementation (93.90%, VERY_HIGH) and a substantially different implementation (41.15%, MEDIUM). Larger labeled evaluation is still required for production calibration.

Appendix A. End-to-End Response Example

```
{
  "success": true,
  "submission": {
    "submission_id": "d111bb1f-995c-4f00-8b84-79059885e6b6",
    "user_id": "b5a7b3d2-f4b2-4d95-8d6d-9d8d2d1f4a11",
    "domain_role_id": "3b297326-908f-4fa3-a010-05e541c7aac9",
    "assignment_id": "550e8400-e29b-41d4-a716-446655440003",
    "submitted_at": "2026-08-22T07:55:01.409Z"
  },
  "plagiarism": {
    "status": "CHECKED",
    "highest_similarity": 98.12,
    "comparison_count": 2
  }
}
```